OMNIA School of Business and Technology

Filière : Technicien Spécialisé – Développement Informatique

RAPPORT DE MINI-PROJET

Intelligence Artificielle & Machine Learning

Backend Intelligent pour la Gestion de Stock

Nom de l'étudiant	Hatim Hanine
Niveau	2e année – Technicien Spécialisé DI
Module	Intelligence Artificielle & Machine Learning
École	OMNIA School of Business and Technology
Encadrant	Mohammed Aitdaoud
Année académique	2025 – 2026

Système de prédiction de rupture de stock basé sur Scikit-learn

Déployé via une API REST Flask

1. Introduction

1.1 Contexte général

La gestion des stocks est un enjeu critique pour toute entreprise commerciale. Une rupture de stock entraîne des pertes de ventes et une insatisfaction client, tandis qu'un sur-stockage génère des coûts inutiles. L'intelligence artificielle offre une solution : prédire automatiquement les besoins avant qu'un problème survienne.

1.2 Problème à résoudre

Les gestionnaires s'appuient souvent sur des seuils fixes qui ignorent les variations saisonnières et les tendances de consommation. L'objectif est d'automatiser ce processus grâce au Machine Learning pour anticiper les ruptures avant qu'elles ne surviennent.

1.3 Objectifs du projet

- Analyser l'historique des ventes et du stock par produit
- Prédire si une rupture de stock est probable dans les 7 prochains jours
- Exposer cette prédiction via une API REST Flask consommable par tout client
- Recommander une quantité de réapprovisionnement optimale

1.4 Intérêt de l'IA

Un modèle ML apprend automatiquement les patterns dans les données historiques et s'adapte aux variations de comportement des ventes, contrairement à une règle fixe. Scikit-learn est simple, bien documenté et parfaitement adapté au niveau 2e année.

2. Présentation du problème métier

2.1 Description du besoin

Un magasin dispose d'un historique de ventes journalières par produit. Chaque jour le stock diminue. La question centrale est : quand passer commande et en quelle quantité pour éviter la rupture ? Le système IA doit répondre automatiquement à ces deux questions.

2.2 Utilisateurs cibles et scénario

- Gérants de commerces, épiceries, responsables logistique en entrepôt
- Développeurs intégrant l'API dans une application ERP ou e-commerce
- Scénario : saisir « PROD-042 » → résultat : « Rupture dans 5 jours, commander 120 unités »

3. Description du jeu de données

3.1 Sources de datasets

- Kaggle – Retail Store Inventory : kaggle.com/datasets/anirudhchauhan/retail-store-inventory-forecasting-dataset
- UCI ML Repository – Online Retail : archive.ics.uci.edu/ml/datasets/Online+Retail
- Option : génération synthétique avec Python (pandas + numpy)

3.2 Variables du dataset

Variable	Type	Description
stock_actuel	Numérique	Quantité en stock ce jour
ventes_jour	Numérique	Quantité vendue ce jour
delai_livraison	Numérique	Délai moyen fournisseur (jours)
produit_id	Catégorielle	Identifiant du produit
rupture_prevue ✓	Binaire (0/1)	Variable cible – Classification
qte_a_commander ✓	Numérique	Variable cible – Régression

4. Prétraitement et analyse des données

4.1 Nettoyage des données

- Suppression des doublons : df.drop_duplicates()
- Valeurs manquantes : df.fillna(df.mean()) pour colonnes numériques
- Incohérences : stock_actuel négatif remplacé par 0
- Conversion de la date : pd.to_datetime(df['date'])

4.2 Encodage et normalisation

- Variable 'produit_id' encodée avec LabelEncoder de Scikit-learn
- StandardScaler appliqué aux features numériques (obligatoire pour KNN/SVM)
- Variable cible 'rupture_prevue' déjà binaire (0/1), aucun encodage requis

4.3 Visualisations recommandées

- Histogramme : distribution du stock journalier par produit
- Scatter plot : ventes_jour vs stock_actuel, coloré par rupture_prevue
- Heatmap de corrélation : variables les plus liées à la rupture de stock
- Boxplot : comparaison du stock selon les catégories de produits

5. Modélisation en Machine Learning

5.1 Type de problème et algorithmes

Problème	Type ML	Algorithme choisi
Prédire la rupture	Classification binaire	**Random Forest Classifier**
Quantité à commander	Régression	**Decision Tree Regressor**

5.2 Justification – Random Forest

- Robuste aux données déséquilibrées (peu de ruptures vs beaucoup de jours normaux)
- Fournit l'importance de chaque variable (feature importance)
- Ne nécessite pas de normalisation, peu sensible aux outliers

5.3 Pipeline ML complet

- Étape 1 : Collecte et chargement des données CSV
- Étape 2 : Nettoyage, encodage, normalisation
- Étape 3 : Séparation train/test (80% / 20%)
- Étape 4 : Entraînement avec model.fit(X_train, y_train)
- Étape 5 : Évaluation avec les métriques adaptées
- Étape 6 : Sauvegarde avec joblib.dump() → stock_model.pkl
- Étape 7 : Chargement dans Flask et exposition via API

6. Entraînement et évaluation du modèle

6.1 Métriques – Classification

Métrique	Description
Accuracy	Pourcentage de bonnes prédictions globales
Precision	Parmi les alertes levées, combien sont réelles ?
Recall	Parmi les vraies ruptures, combien ont été détectées ?
Matrice de confusion	Visualise les faux positifs et faux négatifs

6.2 Métriques – Régression

- MSE (Mean Squared Error) : erreur quadratique moyenne
- RMSE : racine de MSE, dans la même unité que la variable cible
- R^2 : coefficient de détermination — proche de 1 = bon modèle

6.3 Résultats attendus

Un modèle bien entraîné doit atteindre une Accuracy > 85% et un $R^2 > 0.80$. Ces valeurs sont à compléter après expérimentation réelle.

6.4 Code – Entraînement

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import joblib
model = RandomForestClassifier(n_estimators=100, class_weight='balanced')
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
joblib.dump(model, 'stock_model.pkl')
```

7. Intégration dans l'application Flask

7.1 Architecture

L'application est une API REST développée avec Flask (Python). Elle charge le modèle entraîné et expose des endpoints HTTP interrogeables depuis Postman, React ou toute application cliente.

Client (Postman)	→	API Flask /predict	→	Modèle ML (.pkl)	→	JSON résultat

7.2 Endpoints et format JSON

- POST /predict — Reçoit les données produit, retourne la prédiction
- GET /products — Liste tous les produits avec leur état de stock
- GET /health — Vérifie que l'API fonctionne correctement

Entrée : { "stock_actuel": 15, "ventes_jour": 8, "delai_livraison": 3, "produit_id": 5 }

Sortie : { "rupture_prevue": true, "probabilite": 0.87, "qte_recommandee": 120 }

7.3 Code Flask – Endpoint /predict

```
from flask import Flask, request, jsonify
import joblib, numpy as np
app = Flask(__name__)
model = joblib.load('stock_model.pkl')
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    X = np.array([[data['stock_actuel'], data['ventes_jour'], data['delai_livraison']]])
    pred = model.predict(X)[0]
    return jsonify({'rupture_prevue': bool(pred)})
```

8. Plan de réalisation

Étape	Tâche	Détail	Durée
1	**Collecte des données**	Kaggle / génération synthétique Python	3 jours
2	**Préparation**	Nettoyage, encodage, normalisation, visualisations	4 jours
3	**Modélisation**	Entraînement, évaluation, sauvegarde joblib	4 jours
4	**Développement**	API Flask, endpoints, intégration modèle	5 jours
5	**Tests & finalisation**	Tests Postman, débogage, rapport, soutenance	4 jours

9. Analyse critique

9.1 Points forts

- Architecture modulaire : modèle ML séparé de l'API Flask
- API REST standard, compatible avec tout client (web, mobile, Postman)
- Double prédiction (classification + régression) plus utile en pratique

9.2 Limites et difficultés

- Le modèle ignore les événements exceptionnels (fêtes, promotions)
- Déséquilibre des classes : peu de ruptures → utiliser class_weight='balanced'
- Réentraînement manuel requis pour maintenir les performances du modèle

9.3 Améliorations futures

- Tableau de bord Streamlit pour visualiser les prédictions en temps réel
- Réentraînement automatique hebdomadaire avec un scheduler Python
- Déploiement sur cloud (Render, Railway) et connexion à PostgreSQL

10. Conclusion et perspectives

Ce projet a permis de concevoir un backend intelligent capable de prédire les ruptures de stock à partir de l'historique des ventes. Le pipeline complet a été réalisé : de la collecte et du prétraitement des données jusqu'à l'entraînement d'un modèle Random Forest et son déploiement via une API Flask.

Ce travail m'a permis de comprendre comment transformer un modèle de Machine Learning en un outil concret et utile, de gérer des données réelles, d'évaluer objectivement un modèle, et de l'exposer via une interface standardisée. Ces compétences sont directement applicables en entreprise.

Apports du projet

- Maîtrise du pipeline ML complet avec Scikit-learn
- Compréhension de l'intégration entre ML et développement d'application
- Pratique des API REST avec Flask et test avec Postman
- Sensibilisation à l'importance de la qualité des données

Bibliographie / Références

Documentation officielle

- Scikit-learn : https://scikit-learn.org/stable/
- Flask : https://flask.palletsprojects.com/
- Pandas : https://pandas.pydata.org/docs/

Datasets

- Kaggle – Retail Store Inventory : kaggle.com/datasets/anirudhchauhan/retail-store-inventory-forecasting-dataset
- UCI ML Repository – Online Retail : archive.ics.uci.edu/ml/datasets/Online+Retail

Références académiques

- Cours du module Intelligence Artificielle & Machine Learning – OMNIA School
- Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn. O'Reilly.

Annexes

Annexe A – Code complet : Entraînement du modèle

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report
import joblib

df = pd.read_csv('stock_data.csv')
df.dropna(inplace=True)
df['produit_id'] = LabelEncoder().fit_transform(df['produit_id'])

X = df[['stock_actuel','ventes_jour','delai_livraison','produit_id']]
y = df['rupture_prevue']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

model = RandomForestClassifier(n_estimators=100, class_weight='balanced',
random_state=42)
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
joblib.dump(model, 'stock_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
```

Annexe B – Génération synthétique de données

```
import pandas as pd, numpy as np
n = 500
df = pd.DataFrame({
  'produit_id': np.random.choice(['P01','P02','P03'], n),
  'stock_actuel': np.random.randint(5, 200, n),
  'ventes_jour': np.random.randint(1, 30, n),
  'delai_livraison': np.random.randint(1, 7, n),
})
df['rupture_prevue'] = (df['stock_actuel'] < df['ventes_jour'] *
df['delai_livraison']).astype(int)
df['qte_a_commander'] = (df['ventes_jour'] * df['delai_livraison'] * 1.5).astype(int)
df.to_csv('stock_data.csv', index=False)
```